

ACKNOWLEDGEMENT

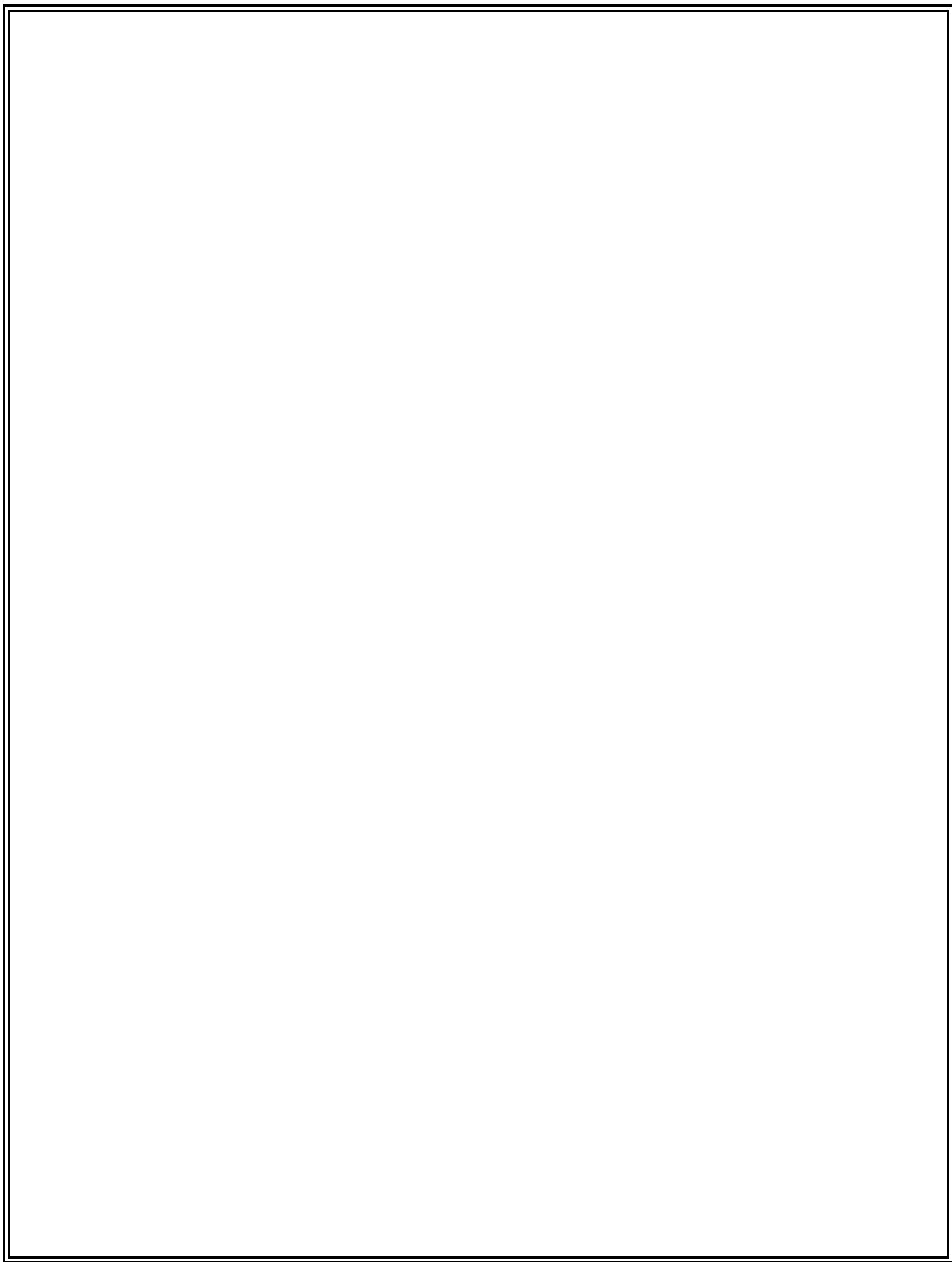
We would like to express our sincere gratitude to **NIRMALA COLLEGE MUVATTUPUZHA** and the Department of **COMPUTER SCIENCE** for giving us the opportunity to undertake this mini project, “**TOURMATE**”, as part of the curriculum.

We are deeply thankful to our respected Principal, **Rev. Fr. Dr. JESTIN K KURIAKOSE**, for providing the facilities and support necessary to complete this work.

We would also like to extend my sincere thanks to **Ms. PREETHY GEORGE**, Head of the Department, for their leadership and support, as well as to all faculty members and staff for their assistance and cooperation.

We are also indebted to our project guide **Ms. TINY THAMPAN**, for their valuable guidance, encouragement, and continuous support throughout the course of this work.

Finally, We wish to acknowledge the constant motivation and support of our family and friends, which greatly contributed to the successful completion of this project.



1.INTRODUCTION

Tourism plays a vital role in today's fast-paced world, offering people opportunities for leisure, exploration, cultural exchange, and relaxation. However, planning a tour often involves several challenges such as finding suitable packages, managing bookings, arranging guides, and creating an itinerary. Traditional methods require approaching travel agencies or relying on scattered online platforms, which can be time-consuming and sometimes confusing.

To overcome these issues, the project '**TourMate**' has been developed as a web-based application that simplifies the process of planning and managing tours. '**TourMate**' is a tour planning website, that aims to provide a convenient, user-friendly and efficient platform that offers an organized and automated approach to tour planning. It provides users with a centralized platform to browse packages, make bookings, and get personalized guidance from assigned tour guides.

1.1 Objective of the project

The primary objective of the '**TourMate**' project is to provide a centralized, user-friendly, and efficient platform for planning tours, booking packages, and receiving guided assistance. The system is designed to reduce the complexities of traditional tour booking and make the entire process more convenient for both customers and administrators.

By integrating tour guide assignment into the system, this project goes beyond simple booking and package browsing. It ensures that each customer receives personalized recommendations and assistance throughout their journey. '**TourMate**' project aims to deliver a reliable and comprehensive platform for tour package management, bookings, and guided travel experiences within a single application.

2.SYSTEM ANALYSIS

System analysis is the process of studying the existing system, identifying its limitations, and designing a new system to overcome those limitations. The analysis of the ‘**TourMate**’ project focuses on understanding the need for an online platform that not only manages tour bookings but also integrates personalized guide assignments

2.1 Existing System

Currently, there are many online travel and tour websites available such as MakeMyTrip, Yatra, Goibibo, Expedia, and TripAdvisor. These platforms allow users to browse different travel and tour packages, make bookings for flights, hotels, and even sightseeing packages. While these websites are widely used and offer multiple services, they also have certain drawbacks:

- **Lack of Personalized Guide Assignment**
Existing platforms do not provide an option where customers are assigned a dedicated tour guide who can assist them throughout their trip.
- **Limited Suggestions from Guides**
While some websites provide generic recommendations or reviews, they do not allow guides to personally suggest places to visit based on customer preferences.
- **Complex Navigation**
Many travel websites are cluttered with multiple services (flights, hotels, cabs, insurance), making it confusing for users who simply want to book a tour package.
- **Dependence on Third-Party Operators**
These websites usually redirect users to third-party agencies, which sometimes causes miscommunication and additional costs.
- **High Service Charges**
Additional booking charges and hidden costs make packages expensive for customers.

- Lack of Admin Control for Custom Packages The admin module in most existing platforms is not flexible for managing customized packages or directly assigning services like a tour guide.

2.2 Proposed System

The proposed system, '**TourMate**', is a web-based application designed to simplify and enhance the tour booking process. Unlike existing travel portals that mainly focus on hotel or flight reservations, this system concentrates on tour packages with integrated guide assignment, ensuring a more personalized travel experience.

Key Features of the Proposed System:

- Customers can browse tour packages easily without unnecessary clutter.
- Online booking of packages with secure database management.
- Admin can add/manage packages, register customers, and assign a tour guide to each booking.
- Guides can suggest personalized places to visit and provide recommendations directly to customers.
- Centralized system ensures smooth communication between admin, customer, and guide.
- Reduced costs since there are no third-party commissions.
- Easy-to-use interface suitable even for users with minimal technical knowledge.

Advantages over Existing Websites:

- Provides tour guide assignment, which current systems lack.
- Offers personalized suggestions from guides instead of generic reviews.
- Focused only on tour planning and packages, making it simpler and less confusing than big portals.
- Direct control for admin to manage users, packages, and guide.

2.3 System Requirement Specification

A software requirements specification (SRS) is a comprehensive description of the intended purpose and environment for software under development. The SRS fully describes

what the colhere will do and how it will be expected to perform. The ‘TravelMate’ is a web-based application developed using PHP and MySQL, hosted on XAMPP. To ensure smooth functioning, the system requires specific hardware and software configurations.

2.3.1 Hardware specification

To host and run the TravelMate web application effectively, the following hardware specifications are required:

Client Side (Users, Admin, Guides)

- Processor: Intel Core i3 or equivalent
- RAM: 2 GB minimum (4 GB recommended)
- Hard Disk: 20 GB free space
- Monitor: 15” Color Display
- Input Devices: Keyboard and Mouse
- Internet Connection: Basic broadband/Wi-Fi

Server-Side (Hosting the Application)

- Processor: Intel Core i5 or higher
- RAM: 8 GB or higher
- Hard Disk: 250 GB or more (for database and backups)
- Network: Stable broadband with static IP (optional for remote access)
- Backup Device: External HDD or Cloud Storage (for safety of database)

2.3.2 Software specification

The software specification details the required software components for developing, deploying and maintaining the TravelMate web application.

Client-Side Software:

- Operating System: Linux
- Browser: Microsoft Edge
- Additional Requirements: JavaScript and Cookies enabled

Server-side software:

- Operating System: Linux Ubuntu (recommended for stability)
- Web Server: Apache (via XAMPP)

- Database: MySQL
- Server-Side Language: PHP
- IDE/Editor: Adobe Dreamweaver / Visual Studio Code
- Additional Software: phpMyAdmin (for database management)

2.3.3 Front End

The front-end is the user interface that allows customers, admins, and guides to interact with the system.

- Languages Used: HTML, CSS, JavaScript
- Description: Provides the user interface for browsing packages, booking tours, admin dashboard, and guide interaction.
- Design Tools: Dreamweaver (for webpage design), Bootstrap .
- Features:
 1. User-friendly design with simple navigation.
 2. Package browsing pages.
 3. Booking forms.
 4. Admin dashboard interface .
 5. Guide suggestion interface.

2.3.4 Back End

The back-end handles business logic, data storage, and communication between the server and database.

- Languages Used: PHP (server-side scripting)
- Database: MySQL (for storing packages, users, bookings, and guide details)
- Server: Apache (via XAMPP)
- Modules/Functions:
 - Admin authentication and management.
 - Customer registration and booking.
 - Package management(add,update,delete packages)

- Tour guide assignment by admin.
- Guide suggestions management.
- Database connectivity query execution.

2.4 Feasibility Analysis

Feasibility analysis is carried out to determine whether the proposed system can be developed and implemented successfully. It evaluates the practicality of the project in terms of technical, operational, and economic factors. Feasibility analysis is performed to determine whether the Tour Planner system can be successfully developed and implemented with the available resources.

1. Technical Feasibility

The 'TourMate' project is technically feasible because:

- The required technologies (PHP, HTML, CSS, JavaScript, MySQL, Apache) are open-source and easily available.
- The system can be developed and run using XAMPP, which provides a complete server environment.
- No special or expensive hardware is required; it runs smoothly on standard personal computers and laptops.
- Tools like Dreamweaver or VS Code can be used for development, both of which are user-friendly.
- The technologies used are widely supported and have strong community support for troubleshooting.

2. Operational Feasibility

The 'TourMate' project is operationally feasible because:

- TourMate provides a user-friendly interface for customers, guides, and admins.
- Customers can easily browse and book packages without needing technical expertise.
- Admin has clear functionalities for managing packages, customers, and guides.

- Tour guides can interact with the system to suggest places, improving customer experience.
- The system reduces manual errors and saves time by automating bookings and guide
- Since the system is web-based, it can be accessed anytime, anywhere with an internet connection.

3. Economic Feasibility

The 'TourMate' project is economically feasible because:

- The software used (PHP, MySQL, XAMPP, Dreamweaver, VS Code) is free or requires minimal licensing cost.
- No expensive hardware is required; the system runs on a standard PC or laptop.
- Maintenance cost is low since it is a database-driven application.
- The system reduces dependency on third-party platforms, thereby saving money for customers.
- In the long run, the system can generate revenue by offering premium packages, advertisements, or subscription-based services.

The Travel Planner system is technically, operationally, and economically feasible. It can be developed and implemented successfully with the available resources, and it provides significant benefits compared to existing systems.

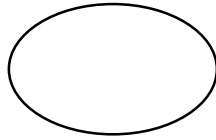
2.5 Data Flow Diagram

2.5.1 Introduction to Data Flow Diagram

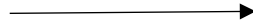
A Data Flow Diagram (DFD) is a graphical representation that illustrates how data flows through a system and how it is processed at different stages. It provides a clear and structured way to analyze and design information systems by showing the movement of data between processes, data stores, and external entities. DFDs are widely used in system design because they are simple, easy to understand, and focus on the flow of information rather than technical implementation details.

Key Components of a DFD

1. **Processes (Circles/Ovals):** Represent the activities or functions where data is processed.



2. **Data Flows (Arrows):** Show the movement of data between processes, data stores, and entities.

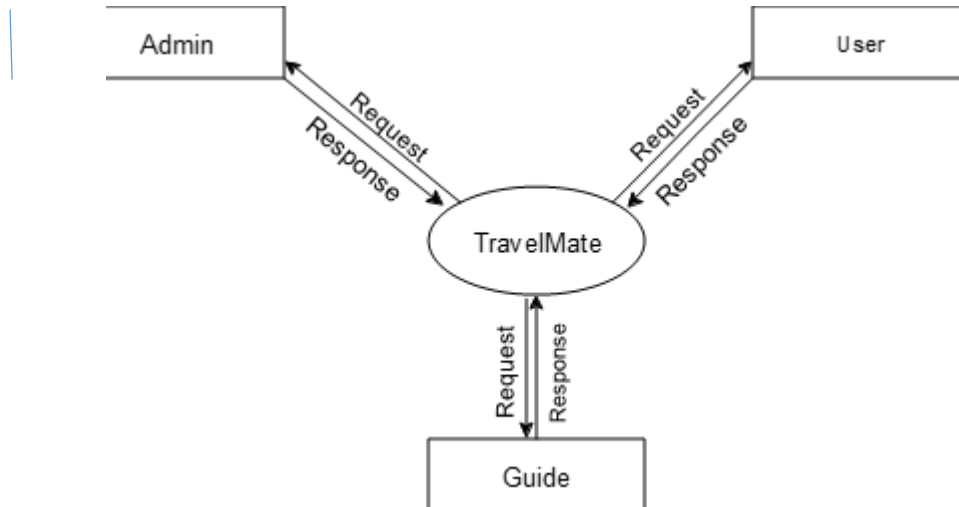


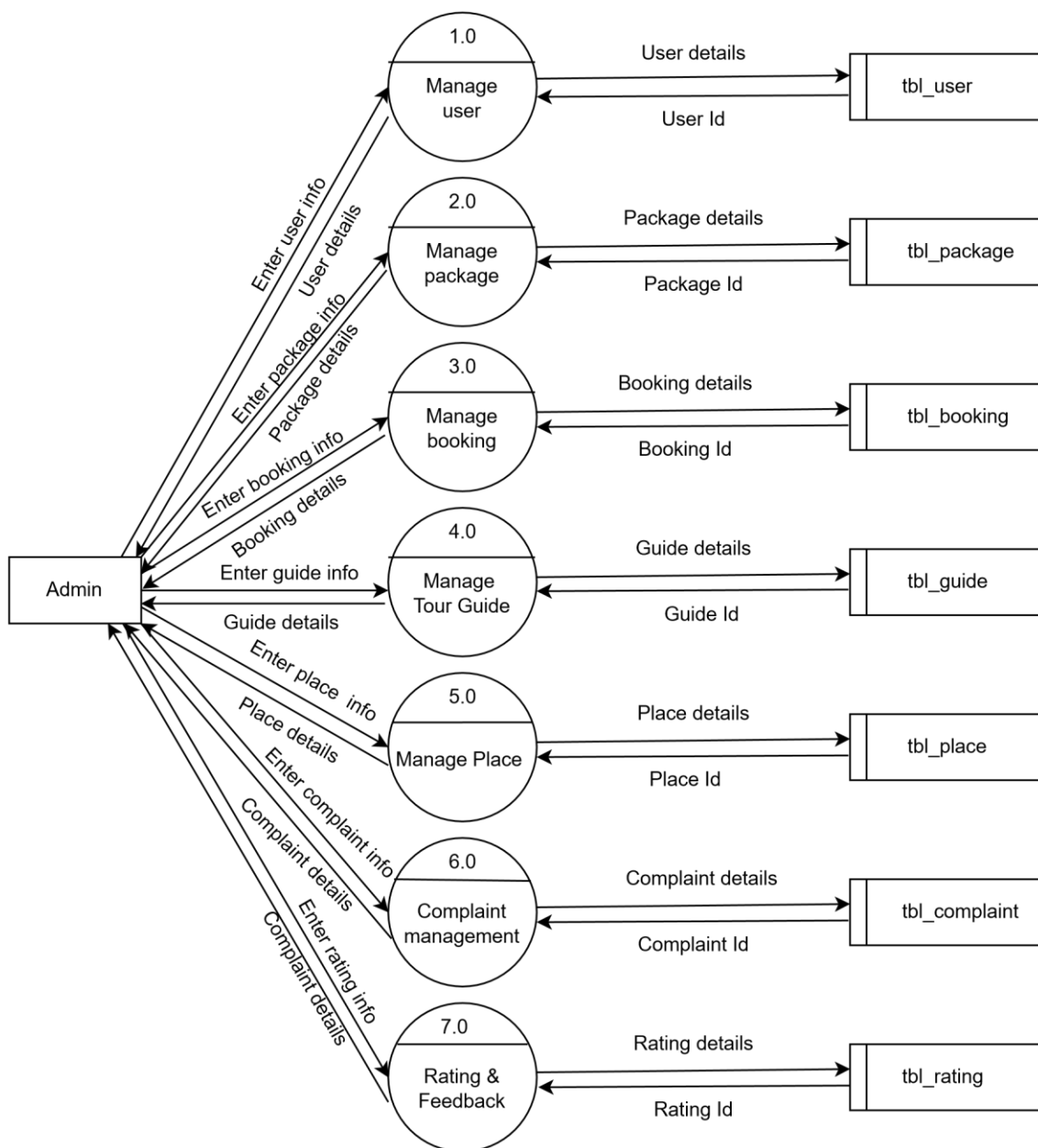
3. **Data Stores (Open Rectangles):** Represent places where data is stored for later use (e.g., databases, files).

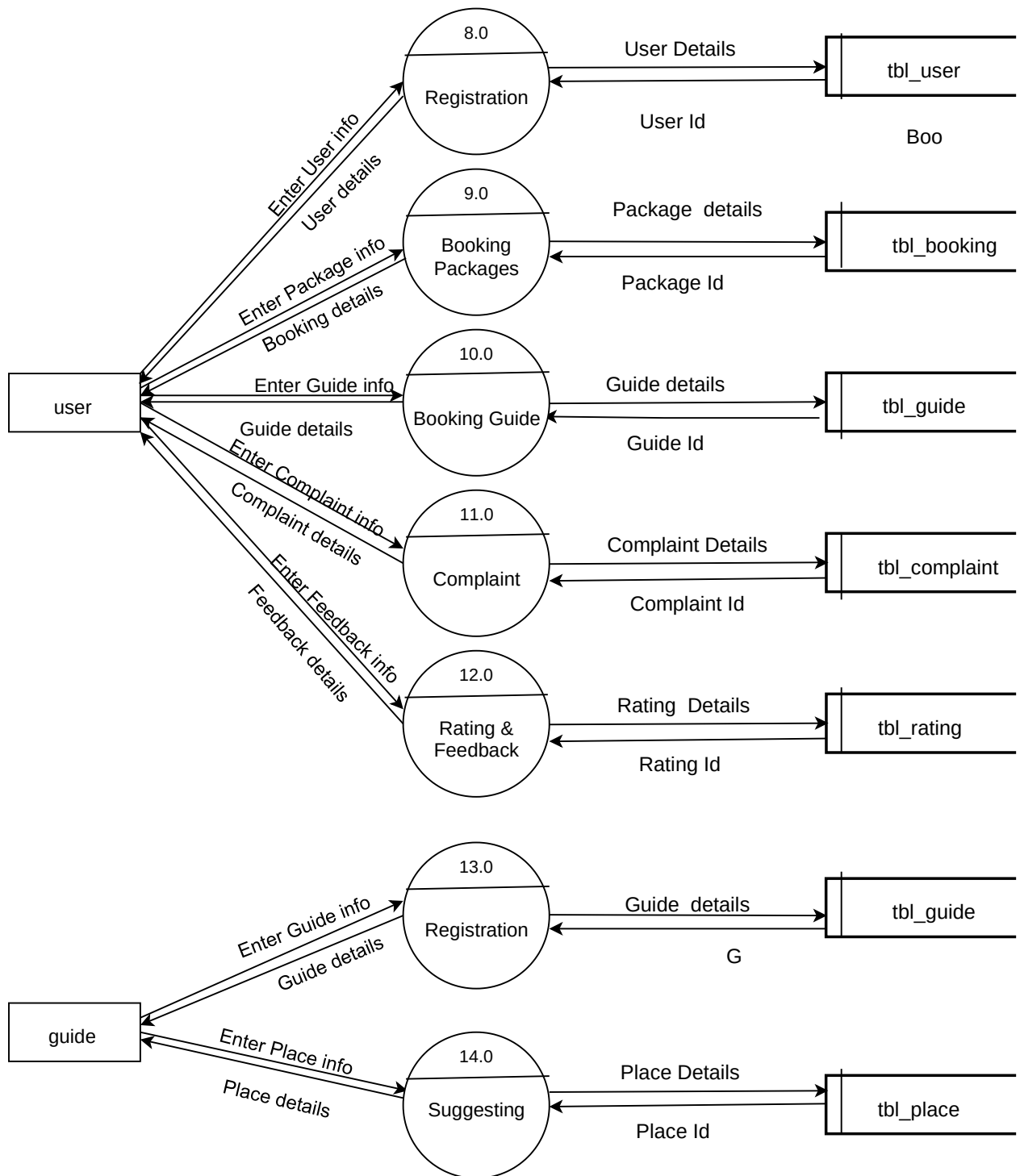


4. **External Entities (Rectangles):** Represent people, organizations, or systems that interact with the application but are outside its boundary (e.g., customers, admins, guides).



LEVEL – 0

LEVEL-1



3. SYSTEM DESIGN

3.1 Input Design

Input Field	Input Type	Example Value	Validation Rules
Full Name	Text Box	John Doe	Cannot be empty, only letters allowed.
Email Address	Text Box (email)	john@example.com	Must be a valid email format.
Phone Number	Text Box (numeric)	+1 234 567 890	10–15 digits, numbers only.
Destination	Dropdown / Auto-complete	Paris, France	Must select from available destinations.
Travel Start Date	Date Picker	2025-12-12	Cannot be a past date.
Travel End Date	Date Picker	2025-12-18	Must be after start date.
Number of Travelers	Dropdown / Stepper	2	Minimum 1, maximum 10.
Travel Type	Dropdown	Adventure	Choose from: Tour, Camp, Devotional.
Budget per Person	Numeric Input / Slider	1000	Positive numbers only.
Special Requests	Multi-line Text Box	Vegetarian meals please	Optional, 250 characters max.

3.2 Output Design

Output Type	Description	Example
Booking Confirmation	Shows booking ID, traveler details, package info, payment.	Booking ID: TPL123456, Paid, PDF download.
Tour Itinerary	Day-wise plan of destinations, activities, and timings.	Day 1: Eiffel Tower, Day 2: Louvre Museum.
Package List	Displays available packages with price and highlights.	Paris Getaway – \$1,000 per person.
Travel Receipts	Payment receipts with taxes and confirmation codes	. Receipt #R78910, PDF available.
Admin Reports	Summaries of sales, destinations, and customer stats.	Top destination: Paris, 200 bookings.
Alerts/Notifications	Sends email or SMS status updates to users.	“Your booking is confirmed!” email alert.

3.3 Table Design

1.tbl_admin

PrimaryKey :admin_id

ForeignKey :

Field	Data Type	Constraints	Description
admin_id	Integer	Primary key	Admin id
admin_name	Varchar(20)	Not Null	Admin name
admin_email	Varchar(20)	Not Null	Admin email
admin_password	Varchar(20)	Not Null	Admin password

2.tbl_district

PrimaryKey :district_id

ForeignKey :

Field	Data Type	Constraints	Description
district_id	Integer	Primary key	District id
district_name	Varchar(30)	Not Null	District name

3.tbl_place

PrimaryKey :place_id

ForeignKey :district_id

Field	Data Type	Constraints	Description
place_id	Integer	Primary key	Place id
place_name	Varchar(20)	Not Null	Place name

district_id	Integer	Foreign key	District id
place_photo	Varchar(60)	Not Null	Place photo
guide_id	Integer	Foreign key	Guide id
place_status	Integer	Foreign key	Place status

4.tbl_package

PrimaryKey :package_id

ForeignKey :type_id

Field	Data Type	Constraints	Description
package_id	Integer	Primary key	Package id
package_title	Varchar(20)	Not Null	Package title
package_detail	Varchar(200)	Not Null	Package detail
package_days	Integer	Not Null	Package days
package_amount	Integer	Not Null	Package amount
type_id	Integer	Foreign key	Type id

5.tbl_packageitem

PrimaryKey :packageitem_id

ForeignKey :place_id,package_id

Field	Data Type	Constraints	Description
packageitem_id	Integer	Primary key	Package item id
packageitem_details	Varchar(200)	Not Null	Package item details
packageitem_name	Varchar(20)	Not Null	Package item name
place_id	Integer	Foreign key	Place id
packageitem_image		Not Null	Package item image
package_id	Integer	Foreign key	Package id

6.tbl_user**PrimaryKey :user_id****ForeignKey :**

Field	Data Type	Constraints	Description
user_id	Integer	Primary key	User id
user_name	Varchar(20)	Not Null	User name
user_email	Varchar(30)	Not Null	User email
user_password	Varchar(20)	Not Null	User password
user_address	Varchar(150)	Not Null	User address
user_photo	Varchar(60)	Not Null	User photo
user_proof	Varchar(60)	Not Null	User proof

7.tbl_booking**PrimaryKey :booking_id****ForeignKey :package_id,guide_id**

Field	Data Type	Constraints	Description
booking_id	Integer	Primary key	Booking id
booking_date	Date	Not Null	Booking date
booking_amount	Integer	Not Null	Booking amount
package_id	Integer	Foreign key	Package id
booking_peoplecount	Integer	Not Null	Booking people count
booking_guidestatus	Varchar(50)	Not Null	Booking guide status
guide_id	Integer	Foreign key	Guide id
booking_fordate	Date	Not Null	Booking fordate

8.tbl_complaint**PrimaryKey :complaint_id****ForeignKey :user_id**

Field	Data Type	Constraints	Description
complaint_id	Integer	Primary key	Complaint id
complaint_title	Varchar(20)	Not Null	Complaint title
complaint_content	Varchar(200)	Not Null	Complaint content
complaint_reply	Varchar(200)	Not Null	Complaint reply
user_id	Integer	Foreign key	User id

9.tbl_feedback**PrimaryKey :feedback_id****ForeignKey :user_id**

Field	Data Type	Constraints	Description
feedback_id	Integer	Primary key	Feedback id
feedback_content	Varchar(200)	Not Null	Feedback content
user_id	Integer	Foreign key	Package detail

10.tbl_type**PrimaryKey :type_id****ForeignKey :**

Field	Data Type	Constraints	Description
type_id	Integer	Primary key	Type id
type_name	Varchar(20)	Not Null	Type name

11.tbl_guide**PrimaryKey :guide_id****ForeignKey :place_id**

Field	Data Type	Constraints	Description
guide_id	Integer	Primary key	Guide id
guide_name	Varchar(20)	Not Null	Guide name
guide_email	Varchar(200)	Not Null	Guide email
guide_password	Varchar(20)	Not Null	Guide password
guide_address	Varchar(50)	Not Null	Guide address
guide_contact	Integer	Not Null	Guide contact
guide_gender	Varchar(10)	Not Null	Guide gender
guide_photo		Not Null	Guide photo
guide_proof		Not Null	Guide proof
guide_status	Varchar(200)	Not Null	Guide status
Place_id	Integer	Foreign key	Place id

12.tbl_rating**PrimaryKey :****ForeignKey :user_id,package_id,guide_id**

Field	Data Type	constraints	Description
rating_content	Varchar(200)	Not Null	Rating content
rating_data			Rating data
rating_datetime	Date	Not Null	Rating datetime
user_id	Integer	Foreign key	User id
package_id	integer	Foreign key	Package id
guide_id	integer	Foreign key	Guide id

4. SYSTEM TESTING & IMPLEMENTATION

4.1 SYSTEM TESTING

System testing is the process of validating the complete and integrated system to ensure that it meets the specified requirements. For the Tour Planner Project, system testing was performed after the successful integration of all modules. The goal was to ensure functionality, performance, security, and user experience were up to the expected standards.

4.1.1 Objectives of System Testing

- Validate that the entire system meets both functional and non-functional requirements.
- Ensure that all integrated modules work together seamlessly.
- Identify and resolve integration issues.
- Verify usability, security, and system reliability.

4.1.2 Scope of Testing

- User Registration & Authentication – Sign-up, login, password reset, and role-based access.
- Tour Package Management – Adding, updating, deleting, and viewing packages.
- Booking & Payment System – End-to-end booking flow, payment processing, and confirmation.
- Search & Recommendation Engine – Accuracy of search results and personalized suggestions.
- Admin Dashboard – Data visualization, reporting, and content management.
- Notifications & Alerts – Email, SMS, and in-app alerts validation.

4.1.3 Types of Testing Conducted

Testing Type	Purpose
Functional Testing	: Verified that all features worked as per the requirements.
Integration Testing	: Ensured proper communication between different modules.
Performance Testing	: Evaluated response times, system stability, and scalability under load.
Usability Testing	: Checked the system's navigation flow and ease of use for users and admins.
Security Testing	: Assessed authentication, authorization, encryption, and data protection measures.
Compatibility Testing	: Tested across various devices, browsers, and operating systems.
Regression Testing	: Retested functionality after bug fixes to confirm no new issues were introduced

4.1.4 Test Environment

Component	Details
Operating System	: Windows
Frontend Technology	: HTML-CSS
Backend Technology	: PHP
Web Server	: Apache(XAMPP)
Browsers for testing	: Google Chrome,Microsoft edge
Database	: MySQL

Testing Tools : Manual testing (Primary), Selenium

4.1.5 Test Results

- All critical features performed successfully during testing.
- Minor issues were identified and resolved.
- Regression testing confirmed system stability after bug fixes.
- The Tour Planner system is now ready for deployment.

4.2 SYSTEM IMPLEMENTATION

4.2.1 System Implementation

System Implementation involves converting the Tour Planner design into a functional system and deploying it in a controlled environment. It ensures all modules are integrated, tested, and accessible to end users.

4.2.2 Implementation Steps:

1. Environment Setup: Install required software, web server, and configure the database.

Example: XAMPP/WAMP with MySQL for database setup.

2. Database Implementation: Create tables, relationships, and populate initial data (users, bookings, tour packages).

3. Front-End Deployment: Upload HTML/CSS files to the web server.

4. Back-End Deployment: Configure PHP scripts and connect them to the database.

5. Payment Gateway Integration: Use the Payment Gateway API to securely process online transactions.

When a user confirms a booking, the system sends transaction details (amount, user info, booking ID) via the API.

The API returns a success/failure response, and the system updates the booking status in the database accordingly.

6. Testing and Debugging: Perform manual testing to ensure all functionalities, including payment processing, work correctly.

7. User Training: Provide guidance for using the system (account creation, searching packages, booking trips).

8. Final Deployment: Deploy the system on a local server or college network for demonstration.

4.2.3 Implementation Environment:

Operating System: Windows 10/11

Front-End Technology: HTML, CSS

Back-End Technology: PHP

Database: MySQL

Web Server: Apache (XAMPP/WAMP)

Browsers for Testing: Google Chrome, Microsoft Edge

Devices: Windows Laptop/Desktop

5.SECURITY TECHNOLOGIES AND POLICIES

The travel planner project implements essential security measures to protect user data:

HTTPS (TLS): All data transmitted is encrypted.

Password Hashing: User passwords are securely stored using bcrypt.

OTP Verification: Users verify accounts via One-Time Password (OTP) for added security.

Input Validation: User inputs are sanitized to prevent SQL injection and XSS attacks.

Access Control: Separate admin and user roles; strong passwords enforced.

Backup & Recovery: Regular backups of critical data are maintained.

These measures provide a basic yet effective security framework suitable for a mini project with limited resources.

6. MAINTENANCE

6.1 Maintenance Activities

- Fixed minor functional bugs identified during testing.
- Performed database backups and cleaned unnecessary test data.
- Ensured security measures, including HTTPS, password hashing, and OTP verification, are functioning.
- Verified input validation to prevent SQL injection and XSS attacks.
- Monitored performance and optimized slow queries.

6.2 Issues Identified

- Updated form validation messages for better clarity.
- Optimized queries to handle occasional slow data retrieval.

6.3 Preventive Measures & Recommendations

- Weekly database and file backups.
- Regular security checks and prompt bug reporting.
- Future improvements: automated testing and additional security/performance enhancements.

7. SCOPE FOR FUTURE ENHANCEMENTS

- The Travel Planner project has wide potential for future growth and scalability. Key areas for enhancement include:
- Adding advanced security features such as two-factor authentication, CAPTCHA, and activity tracking.
- Developing a dedicated mobile application for better accessibility and usability.
- Integrating with real-time flight, hotel, and transport APIs for dynamic data.
- Implementing personalized recommendations, alerts, and notifications to improve user experience.
- Introducing analytics dashboards to track bookings, destinations, and user trends.
- Optimizing performance through database tuning, caching, and load balancing.
- Expanding services to support multilingual interfaces and global destinations.

8. CONCLUSION

TravelMate- the tour planning system has been successfully designed and developed to provide an efficient, user-friendly, and cost-effective solution for managing travel packages, bookings, and tour guide assignments. The system simplifies the process of browsing and booking tours, eliminates the limitations of traditional manual booking methods, and ensures a seamless experience for customers, administrators, and tour guides.

The integration of admin control, guide assignment, and personalized suggestions makes this system unique compared to conventional tour booking websites. Customers benefit from quick access to packages and realtime booking confirmation, while admins can manage users and packages effectively. Tour guides are also actively involved by suggesting places to visit, thereby enhancing the overall travel experience. The project demonstrates the feasibility of using PHP, MySQL, HTML, CSS, Javascript and Bootstrap to build a dynamic web-based application with minimal cost and high scalability.

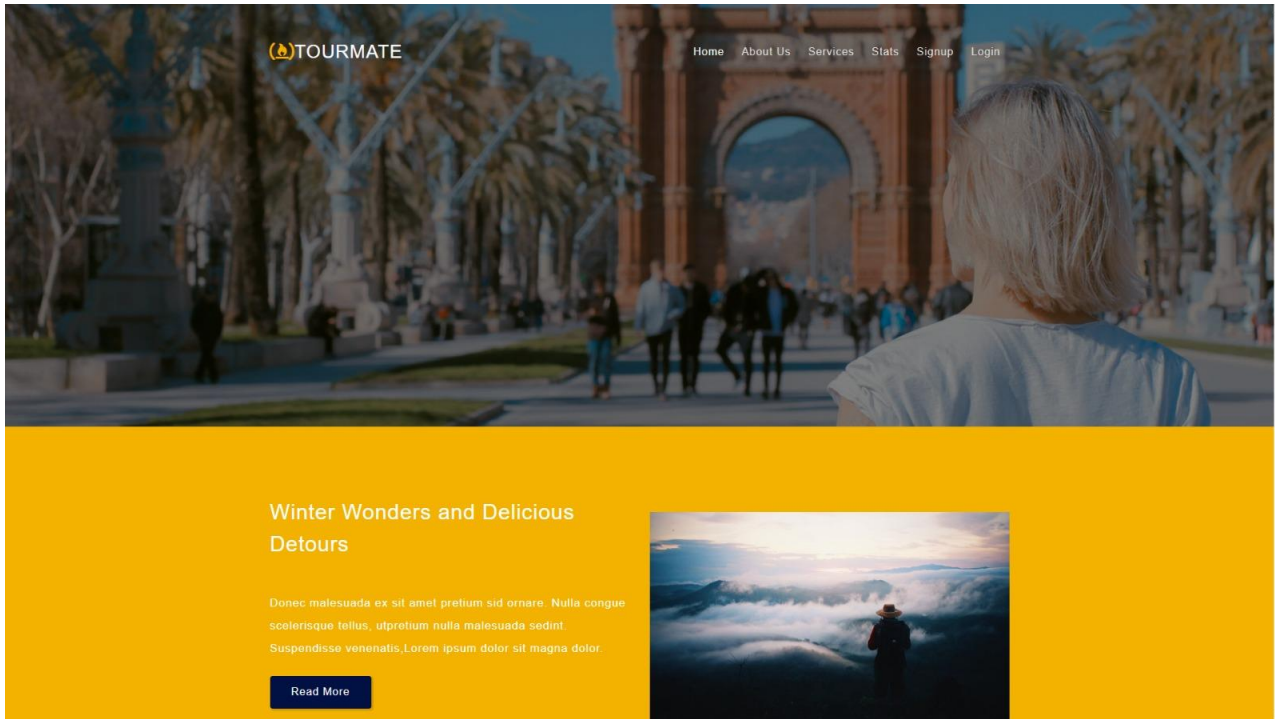
In conclusion, TravelMate- the tour planning system achieves its objective of bridging the gap between customers, travel organizers, and guides, creating a complete digital platform that makes travel planning easier, faster, and more reliable.

9. BIBLIOGRAPHY

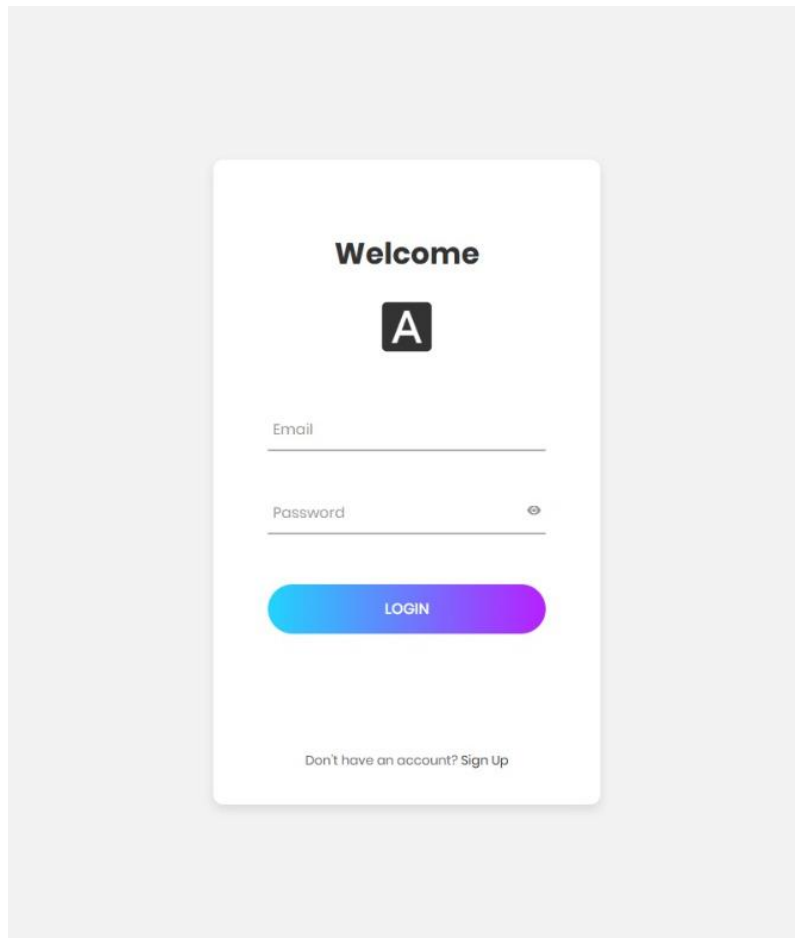
10.APPENDIX

10.1 SCREENSHOTS

Home



Login



A login form UI mockup centered on a light gray background. The form is a white rounded rectangle with a subtle drop shadow. At the top, it says "Welcome" in bold black text, followed by a dark gray square icon containing a white letter "A". Below this are two input fields: "Email" and "Password". The "Password" field has a small eye icon to its right. A large, rounded "LOGIN" button with a blue-to-purple gradient is positioned below the inputs. At the bottom of the form, there is a link that says "Don't have an account? Sign up".

Welcome

A

Email

Password

LOGIN

Don't have an account? Sign up

Sign up

Welcome

Name

Email

Contact

Address

District

Place

No file chosen

Password